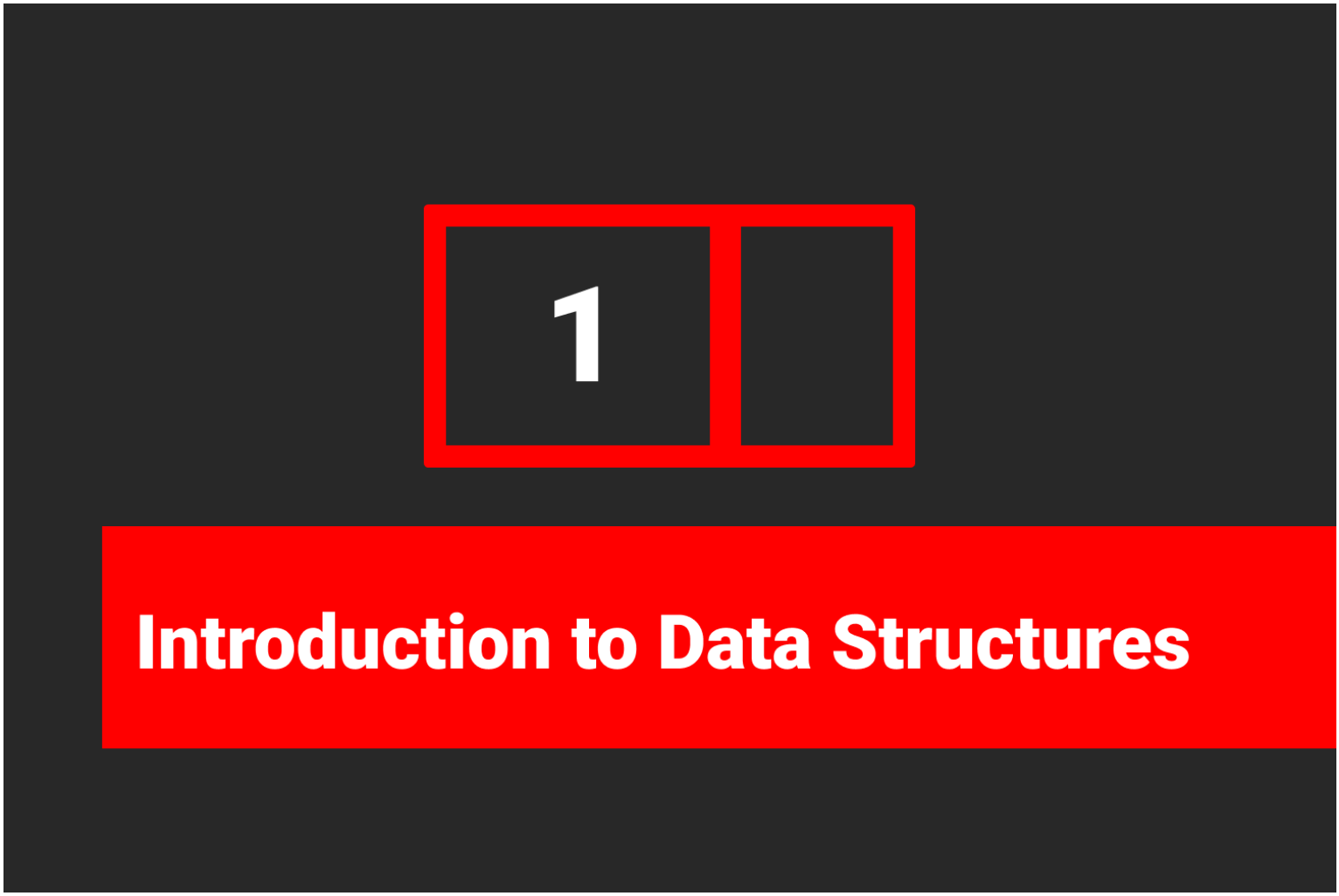
Admin AfterAcademy
19 Jan 2020

Introduction to Data Structures



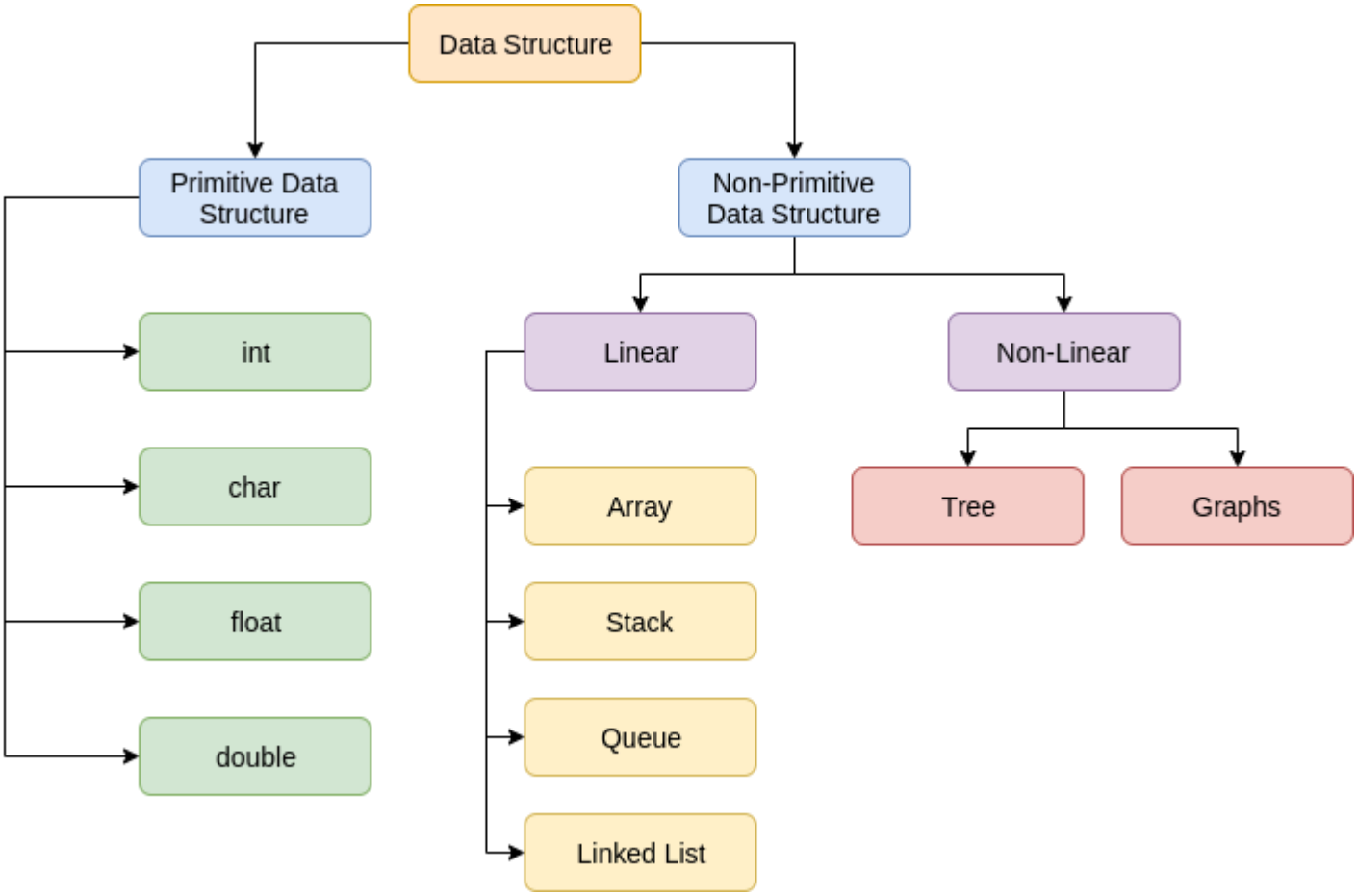
Data structures are the ways of organizing and storing data in a computer so that we can perform several operations efficiently on it. It is widely used in every aspect of computer science. Some examples of commonly used techniques to organize data are:

- Array and Dynamic Array
- Stack and Queue

- Linked List: Singly linked list, Doubly linked lists, Circular list
- Tree: Binary tree, Binary Search Tree, Heap, Trie etc.
- Priority Queue
- Hash Table
- Graph

Data structures are primarily categorized into two parts:

1. Primitive Data Structures
2. Non-Primitive Data Structures



Primitive Data Structure s

These are the predefined way of storing data in the system. All sets of operations are pre-defined. Char, int, float, double are examples of primitive data structures.

Non-Primitive Data Structures

The data structures which are designed using primitive data structures are called non-primitive data structures. They are used to store a collection of data. It can be categorized into two parts:

1. Linear data structure
2. Non-Linear data structure

Linear Data Structure

Data structures in which elements are arranged sequentially or linearly and linked one after another are called linear data structures.

Properties:

- The data elements can be traversed in one time(Single run).
- The elements are adjacently attached in a specified order.
- While implementing this, the necessary amount of memory is declared previously except linked-lists, which does not make good utilization of memory.
- *Examples: Array, Stack, Queues, Linked list.*

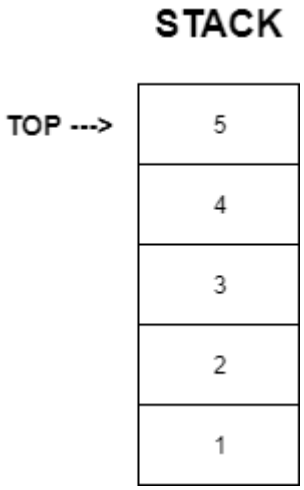
Arrays

An array is a collection of items stored at contiguous memory locations. The idea is to store multiple items of the same type together.



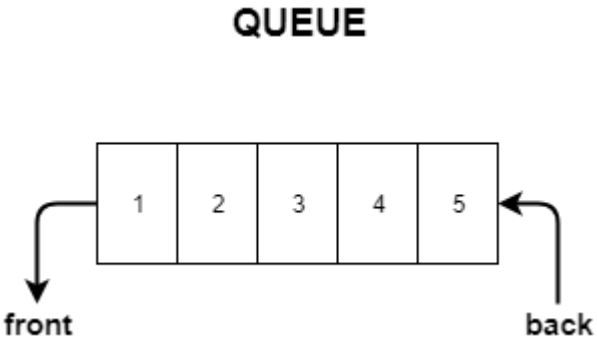
Stacks

The stack is a data structure following the LIFO(Last In, First Out) principle.



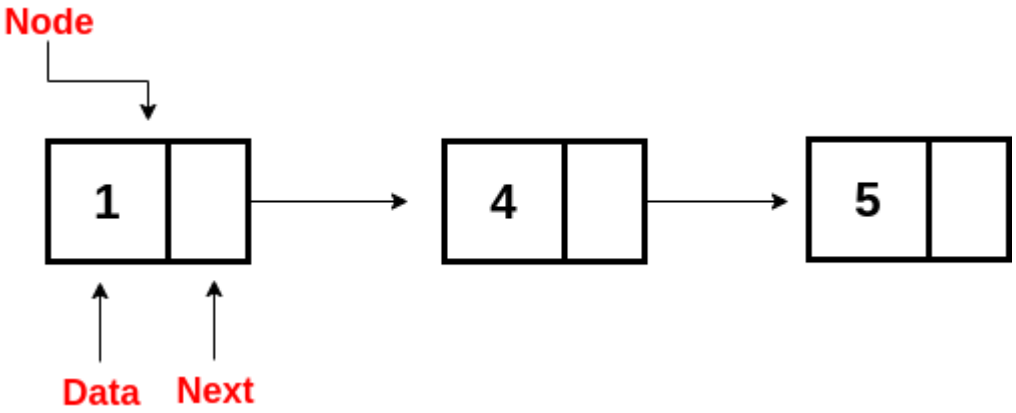
Queues

The queue is a data structure following the FIFO(First In, First Out) principle.



Linked List

A linked list is a dynamic data structure. The number of nodes in a list is not fixed and can grow and shrink on demand.



Non-Linear Data Structure

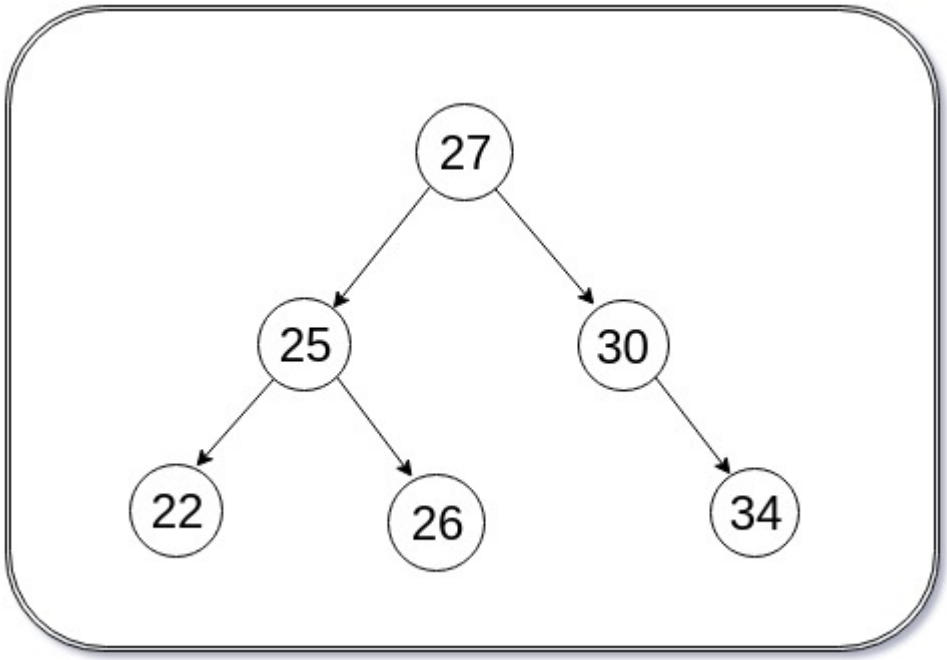
Data structures in which elements are not arranged sequentially are called non-linear data structures.

Properties:

- In these, data elements are attached to more than one element exhibiting the hierarchical relationship between the elements.
- The traversal, insertion, and deletion is not done sequentially.
- It utilizes the memory efficiently and does not require a memory declaration in advance.
- *Examples: Trees and Graphs.*

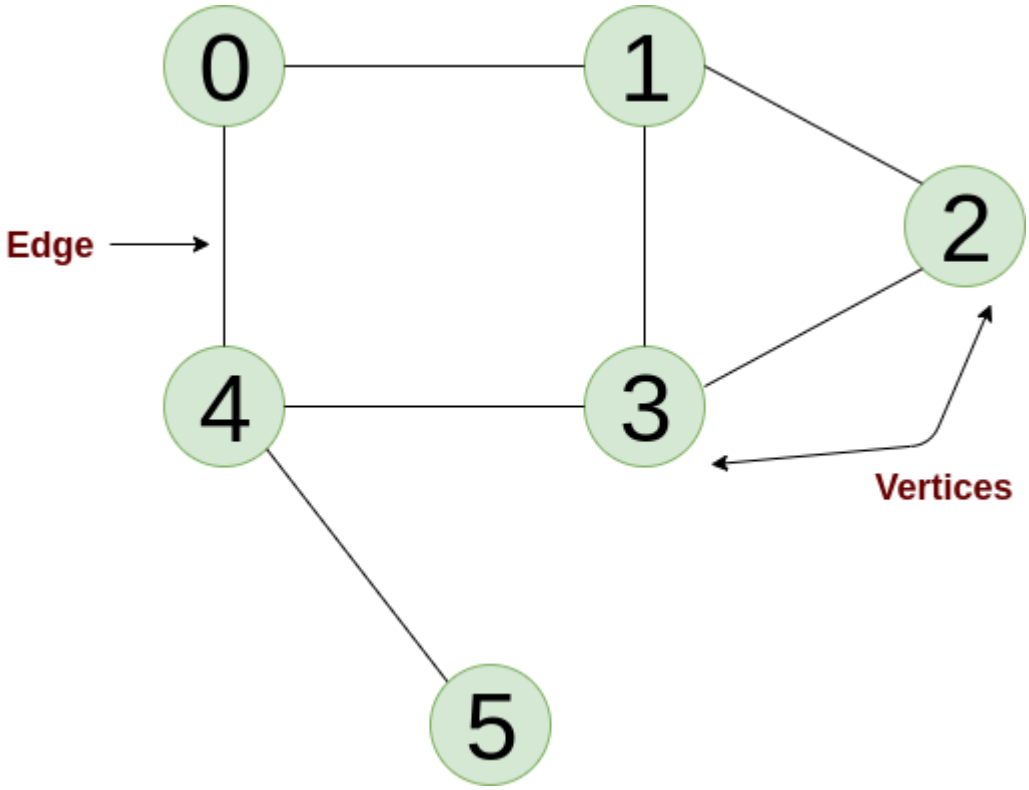
Trees

A tree is a hierarchical data structure that stores the information naturally in a hierarchical style.



Graphs

A graph is a set of objects where some pairs of objects are connected by links. The interconnected objects are represented by points termed as vertices, and the links that connect the vertices are called edges.



Sequential vs Linked Data Structure

We can also classify the data structures based on their representation:

Sequential Representation

Here we allocate a contiguous slot of the memory. This includes the following structures:

- Arrays, Matrices.
- Stack and Queue (Array Implementation)
- Binary tree (Array Implementation)
- Priority Queue (Array and Heap Implementation)
- Hash Table(Open addressing method)
- Graph(Adjacency Matrix representation)

Linked Representation

Here we allocate a distinct chunk of the memory connected via pointers or references. This includes the following structures:

- Linked List
- Binary tree (list implementation)
- Stack and Queue (linked list implementation),
- Priority Queue (Linked list and BST Implementation)
- Hash Table(The Chaining method)
- Graph(Adjacency list representation)

Data Structures as a Dynamic set

In algorithms, the given data set can change over time i.e. it can grow and shrink during the manipulation. So, we refer such collection of data as a dynamic set. The

operation on each dynamic set can be categorized into two parts:

1. **Query Operations:** This will return some info about the dynamic set. For example: Search(S, k), FindMax(S), FindMin(S), FindSuccessor(S, x) etc. are the query operations.
2. **Modifying Operations:** This will change or update the dynamic set. For example, Insert(S, x), Delete(S, x), Merge(S1, S2), Sort(D) etc. are modifying operations.

We can also categories the dynamic set into three categories:

Containers

- Store and process the data independent of the content or the key. We can use Array, Linked list, Stack, Queue, Binary Tree, Graph data structures to implement the container.
- Containers are distinguished by the retrieval order they support. On the basis of the retrieval order, we can divide it into two parts:

1. **Linear:** Array, Linked List, Stack and Queue
2. **Non-Linear:** Binary Tree, Graph, etc.

Dictionaries

- Store and process the data on the basis of key or content. This supports three basic operations on the basis of the key: Search(D, k), Insert(D, k) and Delete(D, k).
- We can use Array, Linked list, BST and Hash Table data structure to implement it. But among all these data structures, the Hash table provides an efficient implementation of the basic dictionary operations. **(Think!)**

Dictionaries Operations	Unsorted array	Sorted Array	Singly Linked List (Un-Sorted)	Singly Linked List (Sorted)	Binary Search Tree	Hash Table
Search(D, k)	O(n)	O(log n)	O(n)	O(n)	O(log n)	O(1)
Insert(D, k)	O(1)	O(n)	O(1)	O(1)	O(log n)	O(1)
Delete(D, k)	O(1)	O(n)	O(n)	O(1)	O(log n)	O(1)

Priority Queues

- Store, access, and process the data in a priority or specific order on the basis of the key value. There are two types of priority queue: 1) Min Priority Queue 2) Max Priority Queue
- The basic priority queue support three operations: Insert(PQ, k), FindMin(PQ) or FindMax(PQ), DeleteMin(PQ) or DeleteMax(PQ).
- We can use Array, Linked List, BST and Heap to implement both min or max priority queue. But among all these data structures, heap provides an efficient implementation of the priority queue. **(Think!)**

Priority Queue Operations	Unsorted array	Sorted Array	Binary Search Tree	Heap
Insert(PQ, k)	O(1)	O(n)	O(log n)	O(log n)
FindMin(D, k)	O(n)	O(1)	O(log n)	O(1)
DeleteMin(D, k)	O(n)	O(1)	O(log n)	O(logn)

Note: Certain dictionary data structure like BST support other useful operations like:

- **Find-Max(D)** or **Find-Min(D)** : Retrieve the item with the largest(or smallest) key from the D. This enables BST to serve as a priority queue also.
- **Predecessor(D, k)** or **Successor(D, k)** : Retrieve the item from D whose key is immediately before (or after) k in sorted order. This enables us to access the element in incremental order.

Application of data structures

- Stacks are used in " ***call stack"*** *which is used when functions are called.*
- Queues are used for interprocess communications.
- Heaps are used to implementing priority queues which are used for scheduler during scheduling of the process in an operating system.
- Binary Search Trees are used for implementing maps and sets.
- B-Trees are used in Database designs
- Trees are used in File-System.
- Circular Doubly linked lists are used by the kernel to store lists of processes.



Janishar Ali

Founder | IIT-BHU | 10 Yrs Exp.



Amit Shekhar

Founder | IIT-BHU | 10 Yrs Exp.

© Copyright 2022, MindOrks Nextgen Private Limited